G1Nav: Humanoid Locomotion RL with Hierarchical Target Navigation

A PPO Walking Policy for the Unitree G1 Driven by a Navigator in MuJoCo

Jeong Hwan Lee

Technical report, status of 5 November 2026

Repository: `physical-ai-robot-learning/g1-target-walking`

***Abstract*—A simulated Unitree G1 humanoid receives an English instruction such as "go to the red cube" and walks to that object in MuJoCo physics. The system is hierarchical: a slow stage selects the target once per episode, a hand-written Navigator turns the target position and the robot's odometry into a velocity command every control step, and a velocity-conditioned walking policy, trained from scratch with PPO in MuJoCo Playground's G1 Joystick environment (MJX), outputs 29 lower-body PD joint targets at 50 Hz. Only the walking policy is learned. In the recorded demos the target is chosen by keyword matching against the simulator's object labels and its position is simulator ground truth; a GR00T N1.6 Eagle vision-language encoder runs on the start ego frame but its output is only logged. The question is therefore the control half: can one walking policy, trained once, be driven by a simple Navigator to instructed targets in closed loop? The policy was trained for 154.8 M environment steps (77 min). Re-evaluated in the raw environment, the first checkpoint that walks is at 41.3 M steps, and the final policy moves at 0.52 m/s for a 0.6 m/s command with a minimum torso height of 0.74 m. In all 3 demo scenes the robot reaches the instructed object (stop radius 0.55 m) without falling; a CPU re-run reproduces the final distances. Getting there required domain randomisation, evaluating without the training wrapper's auto-reset, and two deployment fixes to the observation (normalisation and the frame of the gravity term). Three seeded scenes with one run each are a demonstration, not an evaluation: no success rate is reported.**

I. Introduction

Locomotion policies for humanoids are now routinely trained with reinforcement learning in massively parallel simulation [4], [5]. Such a policy usually tracks a velocity command; something else has to decide where the robot should go. A common structure separates a slow decision about *what* and *where* from a fast controller for balance and stepping. This report describes a small, complete instance of that structure for the Unitree G1 in MuJoCo [1] and what it took to make the two halves work together.

Main question. *Can one velocity-conditioned walking policy, trained once with RL, be driven by a simple Navigator to reach instructed targets in closed loop, with physics only?* The target-selection half is deliberately simple: a keyword rule over the simulator's object labels, with the object's ground-truth position. This report says nothing about whether a vision-language model can do the grounding.

Contributions.

- a velocity-conditioned PPO walking policy for the G1, trained from scratch, with a re-evaluation of every saved checkpoint in the raw environment (Sec. IV-A, V-A);
- a Navigator and a deployment of the trained policy in a different scene, with an observation reproduced term by term from the training environment (Sec. IV-C, IV-D);
- closed-loop target reaching in three seeded scenes, re-measured on CPU from the committed checkpoint (Sec. V-B);
- the failures met on the way and their fixes (Sec. VI).

What is not shown. No learned or visual grounding (the Eagle embedding is never used), no vision in the control loop (this is not a vision-language-action model), no success rate, no new locomotion method (the environment, reward and PPO hyperparameters are MuJoCo Playground's), and no hardware.

II. Related work

Simulation and RL for legged locomotion. MuJoCo [1] and its JAX implementation MJX make thousands of parallel environments practical on one GPU; Brax [2] provides the PPO [3] implementation used here. Rudin et al. [4] showed that massively parallel PPO learns quadruped walking in minutes. MuJoCo Playground [5] packages locomotion environments, including the G1 Joystick task whose environment, reward and hyperparameters this work uses unchanged; its robot model comes from MuJoCo Menagerie [6]. Domain randomisation [7] is part of that recipe and turned out to be necessary here (Sec. VI).

Vision-language backbones for robots. GR00T N1 [8] is a vision-language-action model whose vision-language part is an Eagle backbone. In this project the Eagle backbone of the GR00T N1.6 checkpoint is only run on the start ego frame and its embedding statistics are logged; it was a placeholder for a learned grounding stage that was not built.

III. System

Robot and simulator. The robot is the Unitree G1 model of MuJoCo Playground (from MuJoCo Menagerie) with 29 actuated joints, controlled at 50 Hz with physics at 500 Hz (10 substeps per control step). Training runs in MJX; the closed-loop demos run the same model in plain MuJoCo with the training environment's solver options (Sec. IV-D).

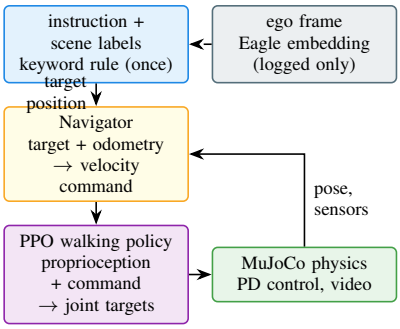


Fig. 1. The hierarchy. Target selection runs once per episode; the Navigator, the policy and the physics run every control step. The Eagle embedding (dashed) does not influence the target.

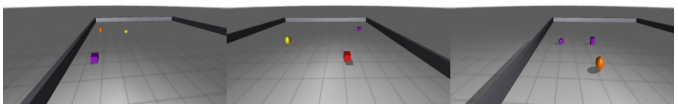


Fig. 2. Start ego frames of the three demo scenes (seeds 0, 1, 2). Every object is in view at spawn.

Scenes. A seed determines a 6 m × 6 m arena with low walls, the robot's spawn at a wall facing inward, and 3 coloured objects (ball, cube, cylinder or cone in one of six colours) at random range and bearing inside the robot's frontal field of view, non-overlapping. The scene carries an ego camera on the torso and a fixed third-person camera (Fig. 2). The recorded demos use seeds 0, 1 and 2.

Observation and action. The policy observes the 103-D state of Playground's G1 Joystick environment: local linear velocity (3), gyro (3), gravity in the pelvis-IMU frame (3), velocity command (3), joint angles relative to the default pose (29), joint velocities (29), last action (29) and the cosine/sine of the gait phase (2). The critic additionally sees a 216-D privileged state. The action is a 29-D vector applied as PD position targets, $q^{\text{target}} = q^{\text{default}} + 0.5\,a$. The command (v_x, v_y, ω_z) is in the robot's heading frame.

Networks (read from the checkpoint). Policy MLP $103 \to 512 \to 256 \to 128 \to 58$ (mean and scale of 29 actions); value MLP $216 \to 512 \to 256 \to 128 \to 1$; a running observation normaliser is stored with the parameters.

IV. Method

A. Locomotion policy

The policy is trained with brax PPO on `G1JoystickFlatTerrain` with 8192 parallel environments, an asymmetric critic and Playground's tuned hyperparameters. Following the official recipe, training uses Playground's domain randomiser and a separate evaluation environment. The final run uses the default reward scales (termination -100, linear-velocity tracking 1.0, stand-still -1.0). At every evaluation a checkpoint is saved and a short rollout with a forward command is run in the raw environment. The run produced 16 checkpoints, 10,321,920 steps apart; the last is at 154,828,800 steps.

Raw-environment evaluation. Every checkpoint is re-evaluated in the registry environment without the brax training wrapper, so that a fall is a fall and not an automatic reset: command $v_x = 0.6$ m/s, 250 control steps (5 s). Reported are the mean local forward velocity (the quantity the tracking reward uses), the net horizontal displacement and the minimum torso height. A checkpoint "walks" if the forward velocity exceeds 0.3 m/s and the torso stays above 0.4 m.

B. Target selection

The instructed target is the object label whose words occur most often in the lower-cased instruction, with ties broken by label order; the target's world position is read from the generated scene. Each demo instruction names a unique colour and shape, so this is a string match, not perception. The Eagle backbone embeds the start ego frame (286 tokens × 2048-d) and the shape and norm of that embedding are logged.

C. Navigator

The scene is static, so the target's world coordinate is fixed once and tracked with odometry (the robot's base pose). Every control step the Navigator computes the bearing and distance to the target and issues a turn-then-go command: the yaw rate is the bearing (rad) clipped to 0.5 rad/s; the forward speed is 0.45 m/s when the bearing is below 40.0° and a quarter of that while turning, capped at 1.2 times the distance; no lateral command. Both channels are slew-rate limited (0.04 m/s and 0.05 rad/s per step), because the policy topples on a step change in command. The robot stops inside 0.55 m.

D. Deployment in the navigation scene

The navigation scene is Playground's G1 scene with the objects, walls and cameras injected. The policy only works there if its inputs match training exactly. Three things had to be reproduced: the physics options (timestep, 3 solver iterations, 5 line-search iterations, Euler integrator without Euler damping), the environment's hard-coded default pose (not the keyframe's) for both the observation and the control, and the observation itself, read from the same named sensors as in training, normalised with the stored running statistics, and with the gravity term computed in the pelvis-IMU frame (Sec. VI). Each episode holds a zero command for 2 s before navigation (and for 2 s after it when the target was reached), ramps the command up over the first 25 control steps, and ends when the Navigator stops, when the torso drops below 0.4 m, or after a 12 s budget.

V. Experiments and Results

All numbers below are taken from committed files: the checkpoint pickles and training metrics in `results/checkpoints/`, the CPU re-measurements in `results/` (made from the committed checkpoints; the original GPU runs' logs are not in the repository), and the demo episode files in `media/`.

TABLE I
WALKING EMERGENCE: EVALUATION REWARD OF TRAINING AND RAW-ENVIRONMENT RE-EVALUATION (COMMAND 0.6 M/S, 5 S).

step	reward	fwd vel	net dist	min z	verdict
0	−6.30	0.36	0.45	−0.78	falls
10.3 M	−2.83	0.28	0.55	−0.78	falls
31.0 M	−2.09	−0.27	0.76	−0.76	falls
41.3 M	**0.03**	**0.32**	**1.54**	**0.73**	**walks**
61.9 M	3.99	0.50	2.51	0.73	walks
144.5 M	8.75	0.58	2.89	0.74	walks
154.8 M	9.29	0.52	2.59	0.74	walks (final)

Velocities in m/s, distances and heights in m. A negative min z is a fallen robot.

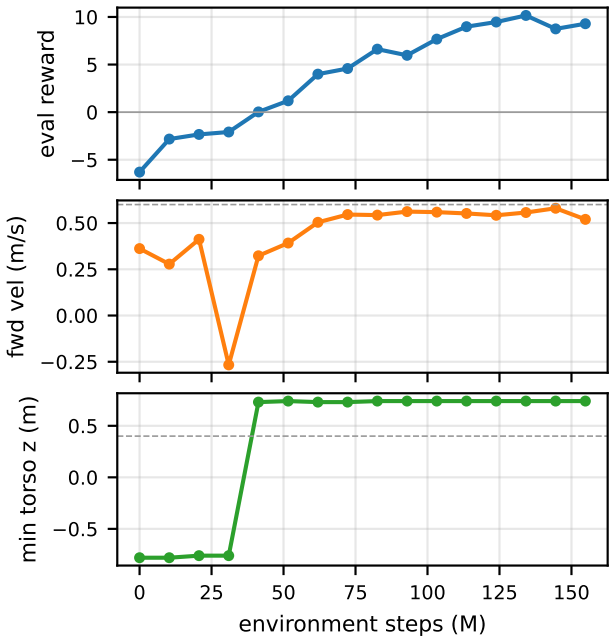


Fig. 3. Per checkpoint: evaluation reward during training (top), and in the raw-environment re-evaluation the mean forward velocity against the commanded value (middle, dashed) and the minimum torso height against the fall threshold (bottom, dashed).

A. Walking emergence

Table I and Fig. 3 show the evaluation reward of training and the raw-environment re-evaluation of the checkpoints. Walking first appears at the 41.3 M checkpoint, where the evaluation reward also crosses zero; from there every checkpoint walks. The highest forward velocity in the sweep is 0.58 m/s at 144.5 M steps; the final policy moves at 0.52 m/s for the 0.6 m/s command (87 %) and covers 2.59 m in 5 s. The training-time rollout of the final checkpoint (4 s) gives 0.519 m/s, 2.07 m and a minimum torso height of 0.745 m. Throughput was about 2.3 M steps/min between the first and the last evaluation and about 2.0 M steps/min over the whole run including compilation.

TABLE II
CLOSED-LOOP DEMO EPISODES (ONE RUN PER SEED).

seed	instruction / chosen target	final dist	torso z	frames
0	orange cylinder	0.39	0.75	630
1	red cube	0.41	0.76	387
2	purple cylinder	0.54	0.75	668

Instruction "go to the ⟨target⟩"; all three reached, none fell. Distances and heights in m; frames of the video.

B. Closed-loop target reaching

Table II lists the three demo episodes. In each, the keyword rule selected the instructed object, and the robot reached it without falling. "Reached" means that the Navigator's stop condition fired (planar distance at most 0.55 m); the final distance is measured after the standing post-roll. Seed 2 stops at 0.54 m against the 0.55 m radius. Re-running the three episodes on CPU reproduces the final distances and torso heights to the printed precision; frame counts match for seeds 0 and 2 and differ by one for seed 1 (386 vs 387). Fig. 4 shows the seed-0 episode.

VI. FAILURE ANALYSIS

The engineering log below describes intermediate runs and code versions whose logs are not in the repository; it is the author's record, not re-checkable results.

Reward rises, the robot does not walk. Without domain randomisation the evaluation reward increased but the policy did not walk. Adding the domain randomiser and a separate evaluation environment made walking learnable.

The "marches in place" illusion. With a softened termination penalty (−3) the policy reached a high reward (+19) by marching in place: with falling almost free, stepping safely collects the air-time, phase, posture and angular-tracking terms without forward motion. The in-training rollout hid the falls, because the training wrapper resets the robot to the start pose on every fall, so a falling policy rendered as stable stepping. Restoring the default termination penalty and evaluating every checkpoint in the raw environment fixed both.

Observation mismatch. Building the observation by hand from positions, velocities and the base quaternion made a walking policy fall; reading the same named sensors as the training environment fixed it.

The closed loop fell over. The policy walked in the raw environment but toppled within about 1.6 s in the navigation scene. Comparing the observation term by term with the training environment found two bugs. (a) Training normalised observations, and the checkpoint stores the running statistics, but the deployment built the network without the normaliser, so the policy saw raw observations; fixing this alone let the robot reach a target straight ahead. (b) The gravity term of training is the world down vector expressed in the pelvis-IMU frame; the deployment used the negated world-frame up vector, which coincides with it only while the robot walks straight. The robot fell on every turn, exactly the symptom. With both fixes all three demo scenes are reached upright.

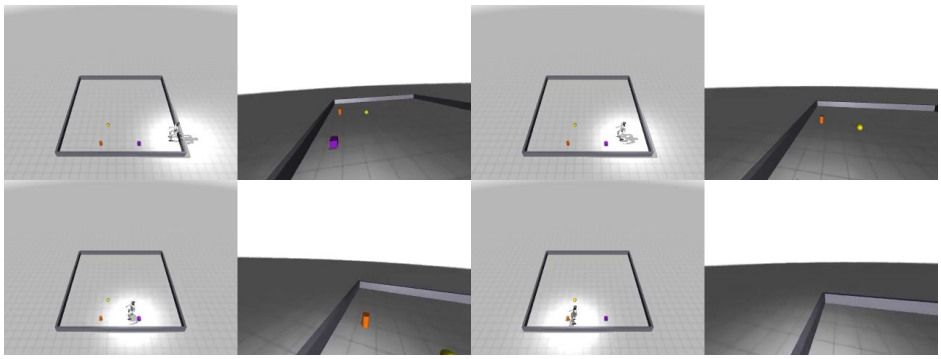


Fig. 4. "go to the orange cylinder" (seed 0): third-person view and ego camera at the end of the standing pre-roll, while turning, while approaching, and standing at the target (left to right, top to bottom).

Infrastructure. The pinned JAX stack failed with a cuSolver internal error on an Ampere GPU and had no kernels for Blackwell; training ran on an RTX 6000 Ada. Rendering a rollout video at every evaluation with too many evaluations throttled training about 50×; 50 evaluations over the run (one every ∼10 M steps) kept it fast.

VII. Limitations

Target selection is keyword matching and the target position is ground truth; instructions must name the object's colour and shape, and nothing is learned or evaluated on the perception side. The results are three seeded scenes with one run each, static objects and flat terrain: a demonstration, not an evaluation, and no success rate is reported. The static-scene assumption fixes the target once. The walking policy follows the Playground recipe and reward; the contribution is the coupling and the debugging. Everything is in simulation; no transfer to hardware was attempted.

VIII. Reproducibility

Layout. All code is one Python package, `src/g1nav`: `envs` (seeded scenes, instruction templates), `locomotion` (PPO training, raw-environment evaluation), `navigation` (keyword rule, Eagle logging, Navigator, closed loop, demo re-run) and `apps` (dashboards). `results/` holds the checkpoints, the training metrics and the CPU re-measurements; `media/` the demo videos, start ego frames and cognition files; `scripts/pod/` the GPU-machine set-up and runners; `configs/envs/` the pinned requirement files; `archive/` unused code.

Commands (after `pip install -e .`):

```
# training (GPU), default reward scales
python -m g1nav.locomotion.train_walk \
  --termination -100 --track_lin 1.0 \
  --stand_still -1.0 --num_evals 50 \
  --out results/checkpoints/walk_t4
# raw-environment sweep of all checkpoints
python -m g1nav.locomotion.walk_eval \
  --sweep results/checkpoints 'walk_t4_step*.pkl'
# one episode, and the three demo episodes
python -m g1nav.navigation.pipeline --seed 0 \
  --instruction "go to the orange cylinder"
python -m g1nav.navigation.rerun_demo_episodes --cpu
```

What needs what. The claims checks and tests need only Python with numpy (`pip install -e ".[test]"`). The closed-loop episodes and the checkpoint sweep run on a CPU with the pinned simulation stack (`jax 0.5.3`, `brax 0.14.2`, `mujoco 3.4.0`, `playground 0.1.0`); an opt-in test re-runs both and compares them with the committed files. Full-scale training needs an NVIDIA GPU. The Eagle encoder needs torch, a CUDA GPU and the backbone, which is not redistributed.

Number checks. Every number in this report is taken from a file in the repository and checked by `analysis/verify_report.py`; every number in the README by `analysis/verify_claims.py`. Both run in CI.

IX. Conclusion

A velocity-conditioned PPO walking policy, trained once from scratch, can be driven by a hand-written Navigator to instructed targets in closed loop in physics simulation: in all 3 demo scenes the G1 reached the instructed object without falling. The hard parts were not the hierarchy but the details: domain randomisation for learning, an evaluation that cannot hide falls, and an observation at deployment that matches training term by term. The open question this report does not answer is the other half: replacing the keyword rule and ground-truth position with perception, and measuring a success rate over many scenes.

References

[1] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ IROS*, 2012, pp. 5026–5033.

[2] C. D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, and O. Bachem, "Brax – A differentiable physics engine for large scale rigid body simulation," in *NeurIPS Datasets and Benchmarks*, 2021.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[4] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proc. Conf. Robot Learning (CoRL)*, 2021.

[5] K. Zakka *et al.*, "MuJoCo Playground," arXiv:2502.08844, 2025.

[6] K. Zakka, Y. Tassa, and MuJoCo Menagerie contributors, "MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo," https://github.com/google-deepmind/mujoco_menagerie, 2022.

[7] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ IROS*, 2017.

[8] NVIDIA *et al.*, "GR00T N1: An open foundation model for generalist humanoid robots," arXiv:2503.14734, 2025.